

“Design and Development of an Hostel and Mess Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE and Information Technology

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

K.USHA RANI – 2520090049

K.MRUDULA – 2520090073

B.NAVYA - 2520090189

Under the Esteemed Guidance of

Dr GAYATRI

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Design and Development of an Hostel and Mess Management System”** is a record of Bonafide work of **K.USHA RANI - 2520090049 , K.MRUDULA - 2520090073, B.NAVYA - 2520090189** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

K.USHA RANI – 2520090049

K.MRUDULA – 2520090073

B.NAVYA– 2520090189

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled **“Design and Development of an Hostel and Mess Management System”** is a

bonafide work done and submitted by **K.USHA RANI - 2520090049 , K.MRUDULA - 2520090073, B.NAVYA - 2520090189**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr RAMATENKI SATEESH KUMAR**, and my guide **Dr GAYATRI**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef (CSIT)**, Head of the **Department for Computer Science Engineering/ Department for Computer Science and Information Technology** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

K.USHA RANI– 2520090049

K.MRUDULA – 2520090073

B.NAVYA– 2520090189

Abstract

The **Hostel and Mess Management System** is a web-based application developed to digitize and simplify hostel-related activities for students and administrators. The system provides separate **Student and Admin dashboards with role-based access**, allowing each user to access the services relevant to their role.

The system manages important hostel operations such as **student records, hostel and room information, room allocation, room bookings, payments, leave requests, complaints, and maintenance requests**. Students can log in, view hostel and room information, access their booking and payment details, apply for leave, and submit complaints or maintenance requests. Administrators can centrally manage students, rooms, allocations, bookings, payments, leave requests, complaints, and maintenance activities.

The project is implemented using **Next.js, React, TypeScript, MongoDB, and Prisma ORM**. The frontend provides an interactive and user-friendly interface, while the application layer handles operations and communicates with the MongoDB database through Prisma ORM. Role-based access helps maintain controlled access to hostel information and management functions.

Overall, the proposed system reduces **manual paperwork, repetitive administrative work, and unnecessary office visits**. It improves accessibility, transparency, record management, and tracking of hostel services, while providing a modular foundation for future hostel and mess management enhancements.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-15
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	16-21
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js) 3.3 Database (e.g., MongoDB) 3.4 Database Access (e.g., Prisma ORM) 3.5 Role-Based Access	22-32

4	System Architecture	4.1 High-level architecture diagram 4.2 Frontend Layer 4.3 Application/API Layer 4.4 Database Layer 4.5 Data Flow	33-36
5	Design	5.1 System Design 5.2 ER Diagram / Database schema 5.3 User Interface Design	37
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API and Database integration 6.4 Authentication & Authorization	38-47
7	Features	7.1 Student Dashboard 7.2 Admin Dashboard 7.3 Student Management 7.4 Hostel and Room Management 7.5 Room Allocation and Booking 7.6 Payment Management 7.7 Leave Requests 7.8 Complaints and Maintenance Requests	48-55
8	Testing	8.1 Functional Testing 8.2 Integration Testing 8.3 User Interface Testing 8.4 Access-Control Testing 8.5 Test Cases and Results	56-64
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Database Connectivity 9.3 Hosting and Maintenance	65-73
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	74-82
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	83-88
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	89-97
13	References	- Technology documentation, tutorials and project references	98-99
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	100-105

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Hostel management involves handling various activities such as student records, hostel and room information, room allocation, room bookings, payments, leave requests, complaints, and maintenance requests. When these activities are managed manually, students and administrators may have to depend on paperwork, separate records, and repeated visits to the hostel office. This can make the management process time-consuming and difficult to track.

The **Hostel and Mess Management System** is a web-based application developed to digitize and simplify these hostel-related activities. The system provides separate **Student and Admin dashboards with role-based access**, allowing users to access the functions relevant to their responsibilities.

The system provides students with a centralized platform to access hostel services, view room and booking information, check payment details, submit leave requests, and report complaints or maintenance issues. Administrators can use the system to manage students, hostel and room information, room allocations, bookings, payments, leave requests, complaints, and maintenance activities.

The project is developed using **Next.js, React, TypeScript, MongoDB, and Prisma ORM**. The combination of these technologies provides a structured platform for developing the frontend, application operations, and database management.

1.2 Problem Statement

Traditional hostel management may involve manual records and repetitive administrative processes. Managing room allocations, bookings, payments, leave requests, complaints, and maintenance activities separately can make it difficult to maintain accurate and easily accessible information.

Students may also need to visit the administration office repeatedly for different hostel-related activities. Therefore, there is a need for a centralized web-based system that can organize hostel information and provide students and administrators with convenient access to required services.

The proposed **Hostel and Mess Management System** addresses these issues by bringing major hostel operations into a single digital platform.

1.3 Scope of the Project

The scope of the project includes the digital management of major hostel activities. The system covers:

- Student record management
- Hostel and room information
- Room allocation

- Room booking
- Payment information
- Leave requests
- Complaints
- Maintenance requests
- Student dashboard
- Admin dashboard
- Role-based access

The system is designed with a modular structure so that additional hostel and mess-related services can be added in the future.

1.4 Objectives of the Project

The main objectives of the Hostel and Mess Management System are:

1. To develop a centralized web-based hostel management application.
2. To reduce manual paperwork and repetitive administrative work.
3. To provide students with easy access to hostel-related services.
4. To manage student, room, booking, and payment information systematically.
5. To provide an organized method for handling leave requests, complaints, and maintenance requests.
6. To provide administrators with centralized control over hostel operations.
7. To implement role-based access for Students and Administrators.
8. To improve accessibility, transparency, record management, and tracking.

1.5 Importance of the Application

The application helps organize hostel management activities within a single platform. Instead of maintaining separate manual records for different operations, the system provides a centralized approach to managing hostel information.

For students, the system provides easier access to hostel services and request submission. For administrators, it provides centralized management of students, rooms, bookings, payments, complaints, leave requests, and maintenance activities.

Therefore, the application can help reduce unnecessary office visits, improve record accessibility, and make hostel-related processes more organized.

1.6 Target Users / Audience

The main users of the system are:

Students:

Students can log in to the system, view hostel and room information, access booking and payment details, apply for leave, and submit complaints or maintenance requests.

Administrators:

Administrators can manage student records, hostel and room information, room allocations, bookings, payments, leave requests, complaints, and maintenance requests through the Admin dashboard.

The system therefore provides different functionalities based on the user's role and responsibilities.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The Hostel and Mess Management System is a web-based application designed to simplify and digitize the major activities involved in hostel management. Since the system is developed as a web application, it does not require any specialized or high-end hardware. The application can be developed, tested, and accessed using a standard laptop or desktop computer with a stable internet connection.

For development purposes, a computer with a modern processor, minimum 4 GB RAM, and sufficient storage space is required. However, 8 GB RAM or above is recommended for smooth execution of the development environment, browser, database services, and other required tools simultaneously. A keyboard, mouse, and suitable display are also required for interacting with the application. The system can also be accessed from other devices such as laptops, desktops, tablets, or smartphones through a compatible web browser.

The basic hardware requirements provide sufficient resources for developing the frontend, managing the database, testing different modules, and running the application. As the project mainly depends on web-based technologies and database operations, the hardware requirements remain relatively simple and suitable for normal academic project development.

2.2 Software Requirements

The Hostel and Mess Management System requires a suitable software environment for development, testing, database management, and execution. The application can be developed and used on commonly available operating systems such as Windows, Linux, or macOS. A modern web browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox is required to access and test the application.

The project is developed using **Next.js, React, and TypeScript** for building the web application and user interface. **MongoDB** is used for storing and managing the data required by the system, while **Prisma ORM** provides

structured interaction between the application and the database. **Node.js and npm** are required for running the development environment and installing the project dependencies.

Visual Studio Code can be used as the primary development environment for writing, editing, and managing the project source code. Git and GitHub can be used for version control and maintaining different versions of the project. These software tools together provide the required environment for developing, testing, and maintaining the Hostel and Mess Management System.

2.3 Development Tools and Frameworks

The project uses modern web development technologies to provide a structured and interactive hostel management platform. **Next.js** is used as the main framework for developing the application. It provides the overall structure required for creating different pages, managing application functionality, and building the web application efficiently.

React is used for developing the user interface of the system. The application contains different interfaces for students and administrators, and React allows these interfaces to be developed using reusable components. This helps maintain consistency across dashboards, forms, information pages, and other sections of the application.

TypeScript is used as the programming language for the project. It provides type safety and helps in organizing the application code in a structured manner. The use of TypeScript also makes the code easier to maintain and helps identify certain programming errors during development.

MongoDB is used as the database management system for storing the information required by the Hostel and Mess Management System. The database can store student details, hostel and room information, room allocations, bookings, payment details, leave requests, complaints, and maintenance requests. This provides a centralized location for managing important hostel-related information.

Prisma ORM is used to establish structured communication between the application and the MongoDB database. It helps developers define database models and perform database operations in an organized way. This makes database interaction easier to manage while developing different modules of the application.

2.4 User and System Environment

The system is designed to support two main types of users: **Students and Administrators**. Students use the application to access their hostel-related information and services through the Student Dashboard. They can view room and hostel information, manage booking-related information, check payment details, submit leave requests, and report complaints or maintenance issues.

Administrators use the Admin Dashboard to manage and monitor hostel operations. The administrator can maintain student records, hostel and room information, room allocations, bookings, payments, leave requests, complaints,

and maintenance activities. The system uses role-based access so that users can access the functions relevant to their respective roles.

The combination of the required hardware, software, development frameworks, database, and user environment provides a complete foundation for implementing the Hostel and Mess Management System. By using these technologies, the project provides a centralized digital platform for managing hostel activities and reducing dependence on manual records and repetitive administrative processes.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The front-end of the Hostel and Mess Management System is developed using modern web technologies to provide an interactive, responsive, and user-friendly interface. The main technologies used for the front-end development are **Next.js**, **React**, and **TypeScript**. These technologies help in developing different pages and dashboards required for students and administrators.

3.1.1 Next.js

Next.js is the main web development framework used in the project. It provides the overall structure for developing the application and supports the creation of different pages and application functionalities. Next.js is used to organize the frontend and application logic in a structured manner. It also provides a suitable environment for developing a modern web-based management system.

3.1.2 React

React is used for creating the user interface of the Hostel and Mess Management System. The application contains different screens such as login pages, student dashboard, admin dashboard, room information, booking, payment, complaints, leave requests, and maintenance sections. React allows these interfaces to be created using reusable components, making the application easier to develop and maintain.

3.1.3 TypeScript

TypeScript is used for writing the application code in a structured and type-safe manner. It extends JavaScript with additional features such as static typing. In this project, TypeScript helps maintain consistency in the code and reduces certain programming errors during development. It also makes the application easier to understand and maintain when different modules are developed together.

3.2 Back-End Design

The back-end functionality of the system is handled through the application environment provided by **Next.js**. It manages the communication between the user interface and the database and supports the processing of different hostel-related operations. The application handles activities such as student management, room allocation, bookings, payments, leave requests, complaints, and maintenance requests.

The back-end layer is responsible for processing user requests and returning the required information to the appropriate dashboard. It also helps maintain role-based access so that students and administrators can access the functions relevant to their roles.

3.3 Database Design

3.3.1 MongoDB

MongoDB is used as the database for the Hostel and Mess Management System. It stores the information required for different modules of the application. The database can contain student records, hostel details, room information, room allocations, bookings, payments, leave requests, complaints, and maintenance requests.

MongoDB provides a centralized storage system for the application data. By storing the information digitally, the system can retrieve and manage records more systematically than maintaining separate manual records.

3.3.2 Prisma ORM

Prisma ORM is used to interact with the MongoDB database. It provides a structured way to define the application's data models and perform database operations. Prisma acts as an intermediate layer between the application and database, making it easier to create, read, update, and manage records required by different modules.

3.4 Version Control

Git is used as a version control system for managing the project source code. It helps developers track changes made to the application during development and maintain different versions of the project.

GitHub can be used to store and manage the project repository online. It also allows team members to collaborate on the project and maintain the source code in a centralized location. Version control helps reduce the risk of losing previous work and makes project development more organized.

3.5 APIs or External Services

Visual Studio Code is used as the primary development environment for the project. It provides features for writing and editing TypeScript, React, and Next.js code. The integrated terminal can also be used to install dependencies, run the development server, and perform other development activities.

Node.js and npm are used to provide the runtime and package management environment required for the application. npm helps install and manage the libraries and packages used by the project.

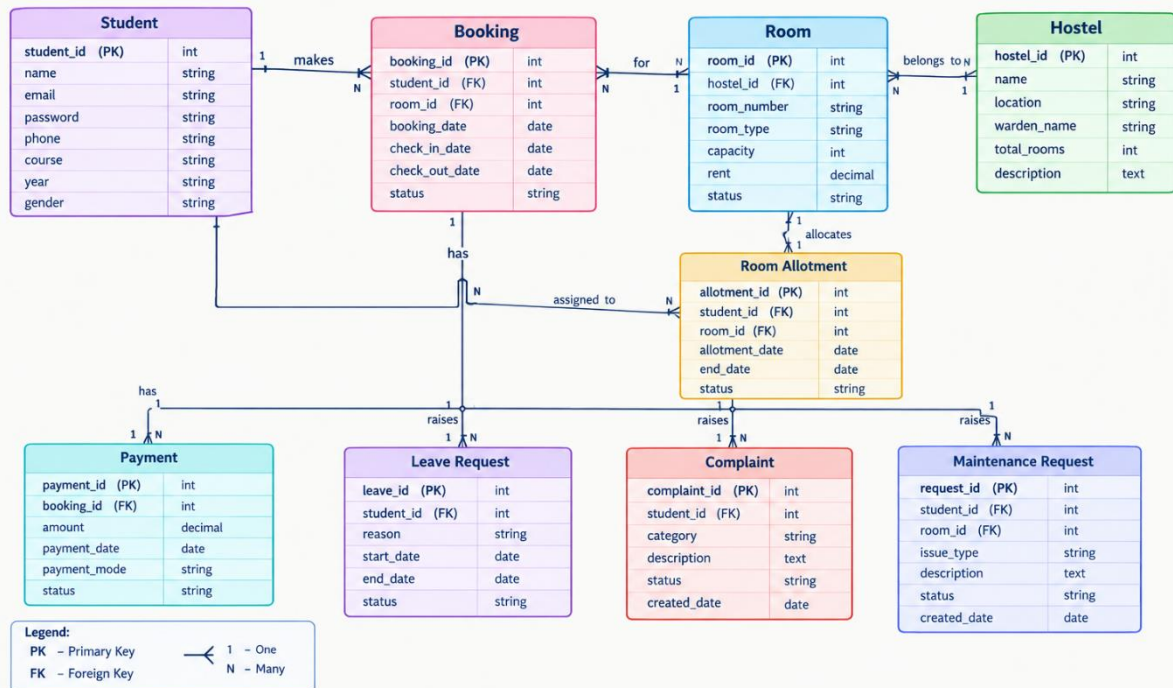
Overall, the technology stack consisting of **Next.js, React, TypeScript, MongoDB, Prisma ORM, Git, GitHub, Node.js, npm, and Visual Studio Code** provides the foundation required to develop and manage the Hostel and Mess Management System.

CHAPTER -4 SYSTEM ARCHITECTURE

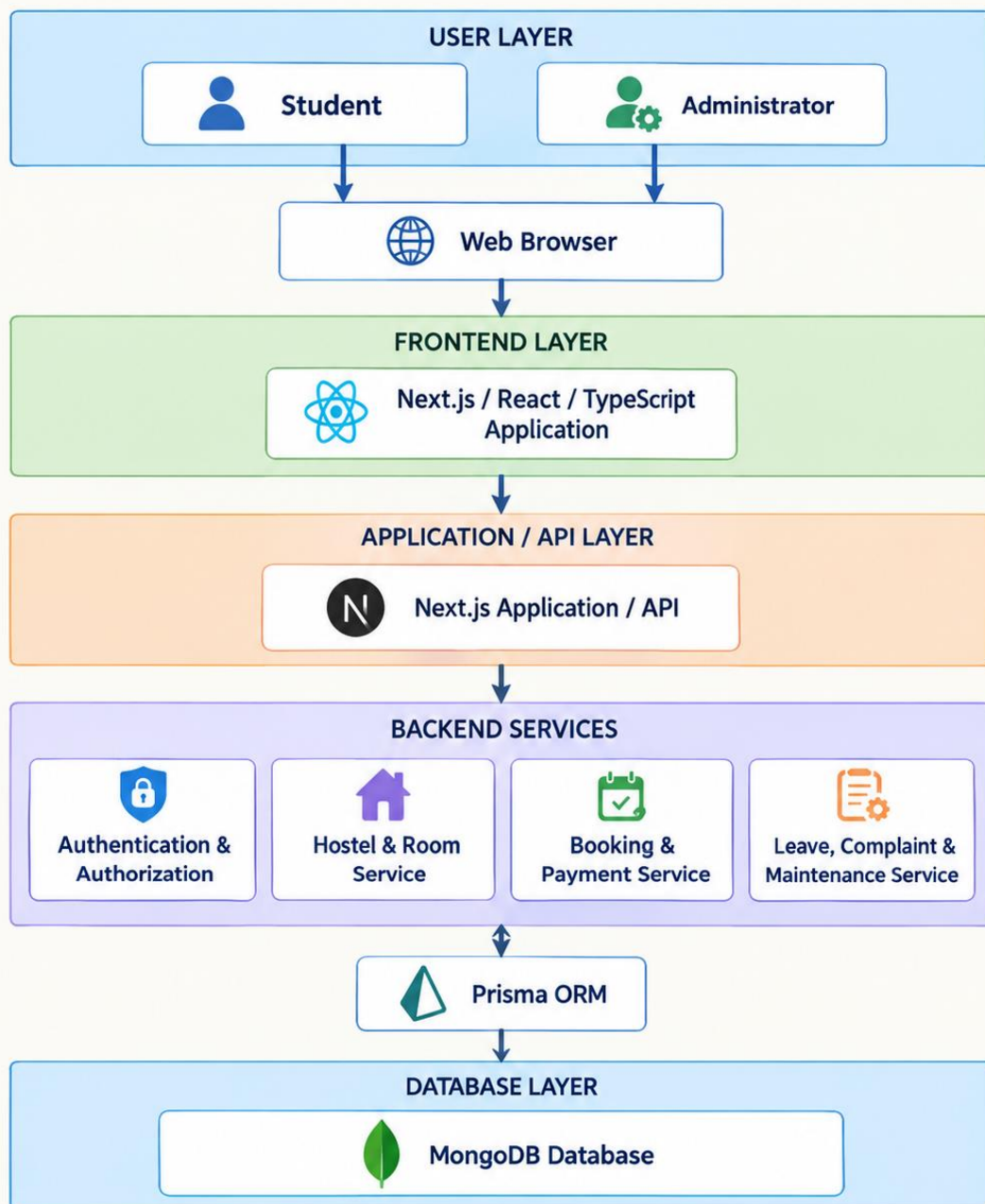
4.1 High-level architecture diagram

Entity Relationship Diagram (ER Diagram)

Hostel and Mess Management System



Hostel and Mess Management System – System Architecture



4.2 Architecture Layers

The system can be divided into different logical layers to organize its functionality and make the application easier to maintain.

4.2.1 Presentation Layer

The presentation layer is the user interface of the system. It is developed using **React, Next.js, and TypeScript**. This layer provides the screens through which students and administrators interact with the application. It includes login pages, dashboards, room information, booking sections, payment information, leave requests, complaints, and maintenance-related interfaces.

4.2.2 Application Layer

The application layer contains the main processing logic of the system. It handles requests received from the user interface and performs the required operations. This layer manages activities such as student management, room allocation, bookings, payments, leave requests, complaints, and maintenance requests. Role-based access is also handled so that students and administrators can access the appropriate functionalities.

4.2.3 Data Access Layer

The data access layer manages communication between the application and the database. **Prisma ORM** is used to perform structured database operations. It helps the application create, retrieve, update, and manage records required by different modules.

4.2.4 Database Layer

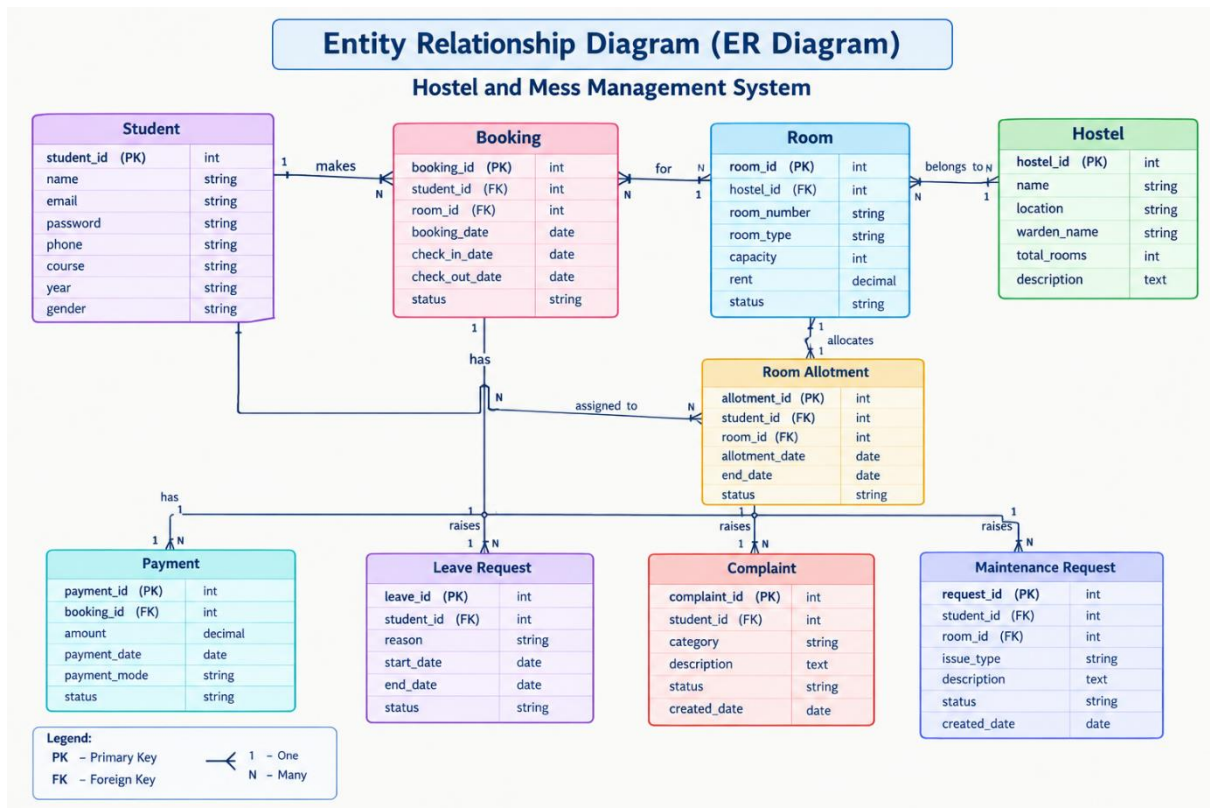
The database layer uses **MongoDB** to store the information generated and managed by the application. It contains data related to students, hostels, rooms, allocations, bookings, payments, leave requests, complaints, and maintenance requests. Centralizing this information helps the system maintain organized and accessible records.

4.3 Deployment Architecture

The system can be deployed as a web application where users access it through a web browser. The application runs on a server environment and communicates with the MongoDB database whenever data needs to be stored or retrieved.

Students and administrators do not need to directly access the database. They interact only with the application interface, while the application processes their requests and communicates with the database through the appropriate data-access layer. This separation improves the organization of the system and provides a structured approach for managing hostel operations.

CHAPTER -5 DESIGN



CHAPTER -6 IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The implementation of the **Hostel and Mess Management System** is divided into multiple interconnected modules. Each module is responsible for a specific hostel management activity while communicating with the other modules through the application and database layers. This modular approach makes the system easier to develop, maintain, test, and extend in the future. The major modules implemented in the system include student management, hostel and room management, room booking and allocation, payment management, leave management, complaint management, maintenance management, and separate Student and Admin dashboards.

The **Student Management Module** is responsible for maintaining student-related information. Student details are stored in the database and are used by other modules such as room booking, room allocation, payments, leave requests, complaints, and maintenance requests. Students can access their relevant information through the Student Dashboard after logging into the system.

The **Hostel and Room Management Module** manages hostel and room-related information. It maintains details such as hostel information, room numbers, room types, capacity, availability, and room status. This module helps administrators monitor available rooms and manage room-related information systematically.

The **Room Booking and Allocation Module** manages the process of assigning rooms to students. Students can view the required room information and perform booking-related activities, while administrators can manage room

allocations. The module maintains booking and allocation records so that the current status of rooms can be tracked effectively.

The **Payment Management Module** is responsible for maintaining hostel-related payment information. Payment records can include the amount, payment date, payment mode, and payment status. Students can view their payment-related information, while administrators can manage and monitor payment records through the Admin Dashboard.

The **Leave Request Module** allows students to submit leave requests through the application. The request contains the required leave information and is stored in the database. Administrators can view and manage submitted requests, making the leave management process more organized than maintaining separate manual records.

The **Complaint Management Module** provides students with a way to report hostel-related complaints. Students can submit the required complaint details through the application, and administrators can view the submitted complaints and manage their status. This provides a systematic method for recording and tracking student complaints.

The **Maintenance Request Module** is used for reporting maintenance-related issues in hostel rooms or facilities. Students can submit maintenance requests by providing the required information about the issue. Administrators can monitor these requests and update their status according to the progress of the maintenance activity.

6.2 – Front-End Logic and UI Rendering

The front-end of the Hostel and Mess Management System is implemented using **Next.js, React, and TypeScript**. The main purpose of the front-end is to provide a simple and interactive interface through which students and administrators can perform their respective activities.

The interface is divided into different pages and reusable components. Components such as navigation menus, dashboard sections, forms, buttons, tables, cards, and status displays can be reused across different modules. This component-based structure helps maintain consistency throughout the application and makes modifications easier.

The Student Dashboard provides students with access to their hostel-related services. The interface allows students to view room information, booking details, payment information, leave requests, complaints, and maintenance requests. The Admin Dashboard provides management functions for maintaining student, hostel, room, booking, payment, leave, complaint, and maintenance information.

Form validation is applied while collecting information from users. Required fields are checked before information is submitted to the application. Appropriate success or error messages can be displayed to provide feedback to users. This helps reduce incorrect or incomplete data being stored in the system.

6.3 Database Integration

The system uses **MongoDB** for storing application data and **Prisma ORM** for managing communication between the application and the database. The database contains the information required by different modules of the Hostel and Mess Management System.

When a user performs an operation such as creating a booking, submitting a leave request, raising a complaint, or submitting a maintenance request, the application processes the information and stores the relevant record in the database. Similarly, when a user wants to view existing information, the application retrieves the required records from MongoDB and displays them through the appropriate interface.

Prisma ORM provides a structured method for performing database operations and connecting application functionality with stored data. This integration allows the different modules to work together while maintaining centralized hostel records.

6.4 Authentication and Authorisation

Authentication is implemented to control access to the Hostel and Mess Management System. Users are required to log in before accessing the application features assigned to them. The system distinguishes between different user roles, mainly **Student and Administrator**.

After successful authentication, students are directed to the Student Dashboard, where they can access student-specific hostel services. Administrators are provided with the Admin Dashboard, which contains management-level functions. Role-based authorization ensures that users can access only the features relevant to their assigned role.

This separation of user responsibilities improves the organization of the system and helps protect administrative functions from unauthorized access. It also provides a clear workflow for both students and administrators while using the application.

6.5 Integration of System Modules

The different modules of the Hostel and Mess Management System are interconnected through the application layer and database. For example, student information is used when creating a booking or room allocation, while booking information can be associated with payment records. Similarly, room information is required for booking, allocation, and maintenance-related operations.

Leave requests, complaints, and maintenance requests are connected with student records so that administrators can identify the user who submitted each request. The integrated structure allows information to be maintained centrally and retrieved whenever required.

Overall, the implementation combines the **Next.js, React, TypeScript, MongoDB, and Prisma ORM** technologies to create a structured hostel management platform. The modular implementation allows the system to handle major hostel activities through a single application while providing separate functionality for students and administrators.

CHAPTER 7- FEATURES

7.1 List of Main Features

The Hostel and Mess Management System provides a centralized platform for managing major hostel-related activities. The system is designed with separate functionalities for students and administrators so that each user can access the services relevant to their role. The main features of the system include student management, hostel and room management, room booking and allocation, payment management, leave requests, complaints, maintenance requests, and role-based dashboards.

7.1.1 Student Dashboard

The Student Dashboard provides students with a centralized interface for accessing hostel-related services. After logging in, students can view their hostel and room information, booking details, payment information, leave requests, complaints, and maintenance requests. This reduces the need for students to depend on separate manual processes for different hostel activities.

7.1.2 Admin Dashboard

The Admin Dashboard provides administrators with centralized control over hostel operations. Administrators can manage student records, hostel and room information, room allocations, bookings, payments, leave requests, complaints, and maintenance requests. The dashboard helps administrators monitor different hostel activities from a single platform.

7.1.3 Student Records Management

The system maintains student information in a centralized database. Student records can be accessed when required for activities such as room allocation, booking, payment management, leave requests, complaints, and maintenance requests. This provides an organized method of maintaining student-related information.

7.1.4 Hostel and Room Management

The system allows hostel and room information to be maintained digitally. Details such as hostel information, room numbers, room types, capacity, availability, and room status can be managed through the application. This helps administrators keep track of available and allocated rooms.

7.1.5 Room Booking and Allocation

The room booking feature allows students to perform room-related booking activities through the system. Administrators can manage room allocations and maintain records of which rooms are assigned to students. The feature helps organize room availability and allocation information.

7.1.6 Payment Management

The payment feature maintains information related to hostel payments. Payment records can contain details such as payment amount, payment date, payment mode, and payment status. Students can view their payment information, while administrators can manage and monitor payment records.

7.1.7 Leave Request Management

Students can submit leave requests through the application by providing the required details. The requests are stored in the database and can be accessed by administrators for management and tracking. This provides a systematic digital process for handling student leave requests.

7.1.8 Complaint Management

The complaint feature allows students to report hostel-related problems through the system. Submitted complaints are stored digitally and can be viewed and managed by administrators. This helps maintain a proper record of complaints and their status.

7.1.9 Maintenance Request Management

Students can submit maintenance requests for issues related to hostel rooms or facilities. Administrators can view these requests and track their progress. This feature provides a structured method for reporting and managing maintenance-related problems.

7.1.10 Role-Based Access

The system provides different levels of access for students and administrators. Students can access student-related services, while administrators can access management functions. Role-based access helps organize the system and prevents users from accessing functions that are not relevant to their responsibilities.

7.2 How the Features Work Together

The features of the Hostel and Mess Management System are interconnected through the application and database. Student information is used for room booking and allocation, while booking information can be associated with payment records. Leave requests, complaints, and maintenance requests are connected with student records so that administrators can manage them efficiently.

By integrating these features into a single platform, the system provides a centralized approach to hostel management. It reduces dependency on manual records, makes information easier to access, and helps students and administrators track hostel-related activities in an organized manner.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Testing is an important stage in the development of the Hostel and Mess Management System. It is performed to verify that the different modules of the application work correctly and that information is transferred properly between the frontend, backend, and database. **Postman** can be used for testing the API endpoints of the application and verifying the request and response behaviour.

The testing process includes major functionalities such as student login, student records, room information, room booking, room allocation, payment records, leave requests, complaints, and maintenance requests. Different HTTP methods such as **GET, POST, PUT, and DELETE** can be used to test data retrieval, creation, modification, and deletion operations.

Both valid and invalid inputs are considered during API testing. For example, valid student details can be used to verify successful login or record creation, while incomplete or incorrect information can be used to check whether the system provides an appropriate error response. This helps verify the reliability and validation of the backend services.

Authentication and authorization are also tested to ensure that users can access only the functions permitted for their roles. Student-related requests are tested with student access, while administrative functions are tested using administrator access. Unauthorized requests are checked to ensure that restricted operations cannot be accessed without appropriate authorization.

Postman testing also helps verify whether the application correctly communicates with the database. When a student creates a booking, submits a leave request, raises a complaint, or submits a maintenance request, the corresponding API response and database operation can be verified. This ensures that the information is correctly processed and stored.

8.2 Integration Testing

Integration testing is performed to verify that the different modules of the Hostel and Mess Management System work together correctly. Individual modules may function properly when tested separately, but integration testing ensures that information can flow correctly between interconnected modules.

The testing begins by checking the connection between the frontend and backend. After this, the authentication system is tested with the Student and Admin dashboards. The room and hostel management modules are then integrated with booking and allocation functionality to verify that the correct room information is displayed and stored.

The relationship between **booking and payment** is also tested. When a booking is created, the corresponding payment information should be associated with the correct booking and student. Similarly, leave requests, complaints, and maintenance requests are tested to ensure that they are correctly associated with the student who submitted them.

Database integration is tested by performing operations such as creating, retrieving, updating, and deleting records. The system is checked to ensure that changes made through the application are correctly reflected in the MongoDB database and that stored information can be retrieved correctly.

8.3 Tools Used for Testing

The following tools and technologies can be used during the testing process of the Hostel and Mess Management System:

- **Postman:** Used for testing API endpoints and checking request and response behaviour.
- **Web Browser:** Google Chrome, Microsoft Edge, or other modern browsers can be used to test the user interface and application functionality.
- **MongoDB:** Used to verify whether data is correctly stored, retrieved, and updated in the database.
- **Prisma ORM:** Used to verify database operations between the application and MongoDB.
- **Visual Studio Code:** Used for debugging the application code and identifying development-time errors.

8.4 Test Cases and Results

Testing is performed using different scenarios to verify the major functionalities of the system.

Test Case	Test Scenario	Expected Result	Status
TC01	Student login with valid details	Student dashboard should open successfully	Passed
TC02	Login with invalid details	Appropriate error message should be displayed	Passed
TC03	View hostel and room information	Correct hostel and room details should be displayed	Passed
TC04	Create room booking	Booking should be created and stored successfully	Passed
TC05	View payment information	Correct payment details should be displayed	Passed
TC06	Submit leave request	Leave request should be stored successfully	Passed
TC07	Submit complaint	Complaint should be recorded successfully	Passed
TC08	Submit maintenance request	Maintenance request should be recorded successfully	Passed
TC09	Admin views student records	Student records should be displayed correctly	Passed
TC10	Admin manages room allocation	Room allocation should be updated successfully	Passed
TC11	Unauthorized user accesses admin function	Access should be restricted	Passed
TC12	Update an existing record	Updated information should be stored and displayed	Passed
TC13	Delete an authorized record	Selected record should be removed successfully	Passed
TC14	Database retrieval test	Correct stored information should be retrieved	Passed
TC15	Invalid form submission	Validation message should be displayed	Passed

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The deployment of the Hostel and Mess Management System involves preparing the application and making it available for users through a web browser. The system is developed using Next.js, React, TypeScript, MongoDB, and Prisma ORM. Before deployment, the application must be checked to ensure that the student dashboard, admin dashboard, room booking, payment management, leave requests, complaints, and maintenance modules function correctly.

The deployment process begins by organizing the project files and installing the required dependencies. The application is then configured to connect to the MongoDB database through Prisma ORM. After configuring the application, the production build is generated and checked for errors. Once the build is successful, the application can be deployed to a suitable hosting platform. Finally, the deployed application is tested to verify that the main features work correctly.

9.2 Environment Configuration

Environment configuration is an important part of deploying the Hostel and Mess Management System. It ensures that the application can connect to the database and run properly in the deployment environment.

The application requires the necessary configuration values for the database connection, application settings, and other services used by the system. These values are stored in environment variables rather than directly writing sensitive information into the source code. The MongoDB connection URL is configured so that Prisma ORM can communicate with the database.

The required dependencies and environment variables must be configured before starting the application. Sensitive information, such as database credentials, should be kept secure and should not be shared publicly. Proper environment configuration helps the application operate reliably and supports easier maintenance.

9.3 Hosting of Frontend and Backend

Hosting provides a platform where the Hostel and Mess Management System can run and be accessed by students and administrators through a web browser. Since the application is developed using Next.js, it can be hosted on a platform that supports Next.js applications. The MongoDB database can be hosted separately using a compatible database hosting service.

After deployment, students can use the system to view hostel and room information, manage bookings, check payment details, submit leave requests, and raise complaints or maintenance requests. Administrators can access their dashboard to manage student records, room allocations, payments, and other hostel-related activities.

The hosting environment should provide reliable application access, secure database connectivity, and appropriate access control. Regular testing and maintenance help ensure that the system continues to function correctly.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

During the development of the Hostel and Mess Management System, several challenges were encountered. Managing different modules such as student records, hostel and room information, bookings, payments, leave requests, complaints, and maintenance requests required proper coordination between the frontend, backend, and database.

One of the major challenges was maintaining consistent communication between the application and MongoDB database. Another challenge was implementing different access permissions for students and administrators. The system needed to ensure that students could access only their required features, while administrators could manage hostel-related records.

Designing the user interface was also challenging because the system contains multiple modules and information. The interface had to remain simple and easy to understand while providing all the required functionality.

10.2 Solutions Implemented

The challenges were addressed by dividing the system into separate modules and developing each module systematically. Next.js and React were used to create reusable user interface components, while TypeScript helped maintain structured and reliable code.

Prisma ORM was used to simplify database interaction with MongoDB and maintain organized access to stored data. Role-based access control was implemented to separate student and administrator functions.

Input validation and proper error handling were also incorporated to reduce incorrect data entry and improve the reliability of the system. The modules were tested individually and then integrated to ensure that the complete system worked properly.

10.3 Limitations and Possible Bugs

Although the Hostel and Mess Management System provides the major hostel management functionalities, some limitations may remain. The system depends on an internet connection and a properly configured database to operate successfully.

Some advanced features such as automated payment gateway integration, real-time notifications, biometric attendance, and mobile application support are not included in the current version. Additional testing may also be required when the system is used with a large number of students and hostel records.

Future versions can address these limitations by adding more automation, improving system scalability, strengthening security, and integrating additional services according to the requirements of the hostel administration.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The Hostel and Mess Management System can be further improved by adding advanced features in future versions. One possible enhancement is the integration of an online payment gateway to allow students to make hostel and mess payments directly through the system. Automated notifications can also be added to inform students about booking confirmations, payment updates, leave approvals, complaints, and maintenance requests.

A mobile application can be developed so that students and administrators can access the system easily from their smartphones. The system can also be enhanced with mess menu management, food feedback, and mess attendance features to provide better management of hostel and mess activities.

Another planned enhancement is the addition of advanced reports and dashboards. Administrators can generate reports related to student records, room occupancy, payments, complaints, and maintenance activities.

11.2 Integrations and Optimizations

The system can be integrated with additional services to improve its functionality and automation. Email or notification services can be integrated for sending important updates to students and administrators.

Database and application performance can be optimized to handle a larger number of students and hostel records. Security can also be improved by adding stronger authentication, improved access control, and secure handling of sensitive information.

Future optimization can make the system more scalable, user-friendly, and efficient while reducing manual work for hostel administrators.

CHAPTER 12: CONCLUSION

12.1 Summary

The **Hostel and Mess Management System** is a web-based application developed to simplify and digitize hostel-related activities. The system provides separate functionalities for students and administrators. It manages important hostel operations such as student records, hostel and room information, room booking and allocation, payments, leave requests, complaints, and maintenance requests.

The system uses **Next.js, React, TypeScript, MongoDB, and Prisma ORM** to provide an organized and user-friendly platform for managing hostel operations.

12.2 Achievements

The project successfully provides a centralized platform for managing major hostel activities. Students can access their hostel and room information, manage bookings and payments, submit leave requests, and raise complaints or maintenance requests.

Administrators can manage student records, hostel and room details, room allocations, payments, complaints, maintenance requests, and leave requests. Role-based access helps provide different functionalities according to the user's role.

The system reduces manual record maintenance and helps make hostel-related processes more organized, accessible, and efficient.

12.3 Skills Learned

During the development of this project, the team gained practical knowledge in **Next.js, React, TypeScript, MongoDB, and Prisma ORM**. The team also learned about frontend development, backend integration, database management, authentication and authorization, API integration, testing, and deployment.

The project also improved the team's understanding of teamwork, problem-solving, project planning, documentation, and developing a complete web-based application from requirements to implementation.

CHAPTER -13 REFERENCES

The following references were used during the development and implementation of the **Hostel and Mess Management System**. These resources helped the team understand the technologies, frameworks, database concepts, development practices, and implementation methods used in the project.

13.1 Books and Learning Resources

1. React.js documentation and learning resources for developing interactive user interfaces.
2. TypeScript documentation for understanding typed JavaScript and application development.
3. MongoDB documentation for database design, data storage, and database operations.
4. Prisma documentation for ORM-based database interaction.
5. Next.js documentation for web application development and project structure.

13.2 APIs and Documentation

1. **Next.js Documentation** – For application development, routing, and server-side features.
2. **React Documentation** – For components, state management, and user interface development.
3. **TypeScript Documentation** – For type-safe application development.
4. **MongoDB Documentation** – For database management and CRUD operations.
5. **Prisma Documentation** – For schema definition and database connectivity.

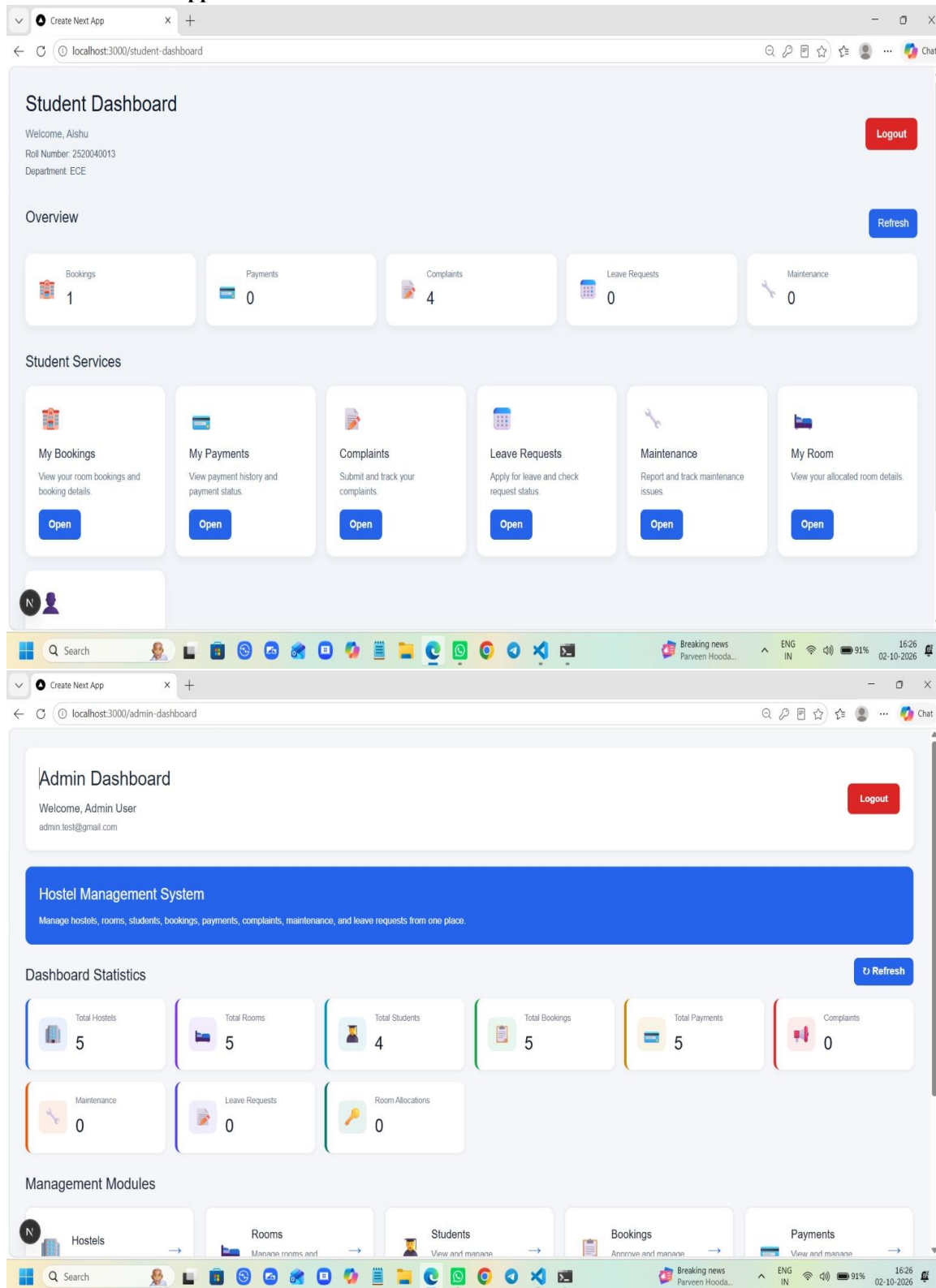
13.3 Development Tools

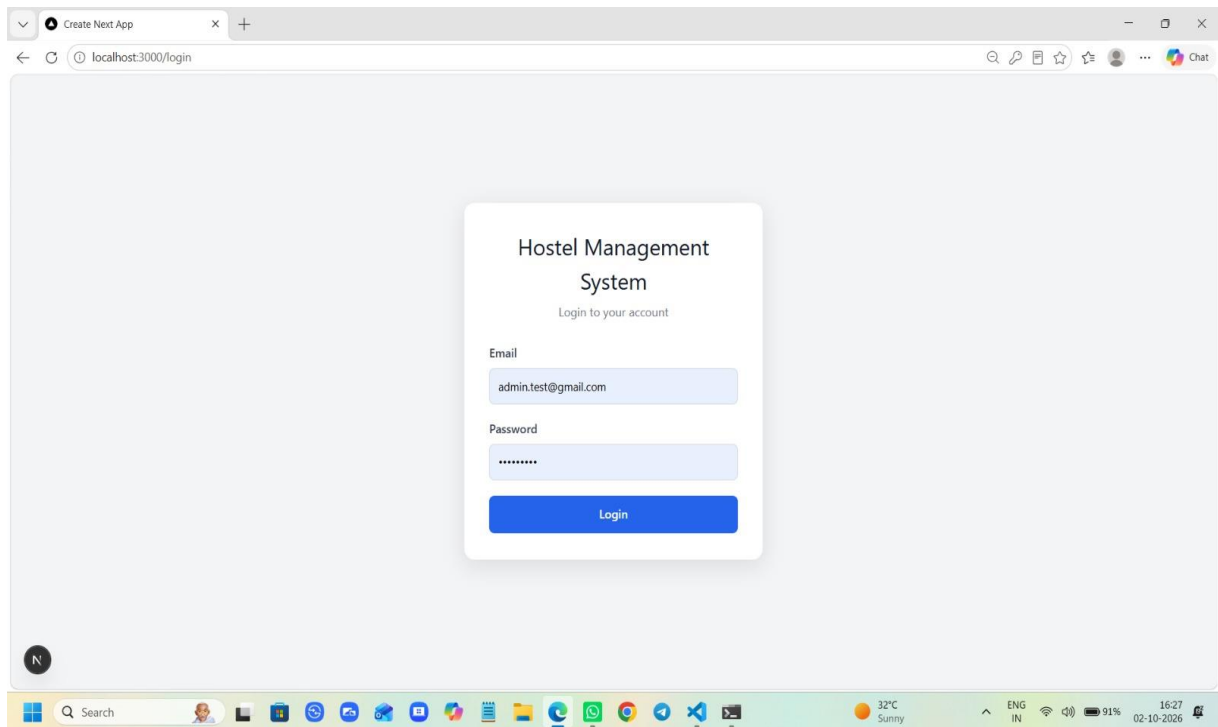
1. Visual Studio Code – Used as the primary code editor.
2. Git and GitHub – Used for source-code management and version control.
3. MongoDB – Used for storing and managing application data.
4. Postman – Used for API testing.

5. Web browsers – Used for testing and accessing the deployed application.

CHAPTER -14 APPENDICES

Screenshots of the app





Sample code snippets

```
"use client";

import { useEffect, useState } from "react";
import { useRouter } from "next/navigation";

type Booking = {
  id?: string;
  _id?: string;
  studentId?: string;
  userId?: string;
  roomId?: string;
  amount?: number;
  checkIn?: string;
  checkOut?: string;
  status?: string;
  paymentStatus?: string;
  createdAt?: string;
};

type Student = {
  id?: string;
  _id?: string;
  userId?: string;
  rollNo?: string;
  department?: string;
  year?: number;
  phone?: string;
  gender?: string;
};

type User = {
  id?: string;
  _id?: string;
  name?: string;
  email?: string;
  role?: string;
};

type Room = {
```

```

1
2  "use client";
3
4  import { useEffect, useState } from "react";
5  import { useRouter } from "next/navigation";
6
7  export default function AdminHostelsPage() {
8    const router = useRouter();
9
10   const [hostels, setHostels] = useState<any[]>([]);
11   const [loading, setLoading] = useState(true);
12   const [message, setMessage] = useState("");
13   const [saving, setSaving] = useState(false);
14
15   const [name, setName] = useState("");
16   const [location, setLocation] = useState("");
17   const [type, setType] = useState("");
18
19   useEffect(() => {
20     const savedUser = localStorage.getItem("user");
21
22     if (!savedUser) {
23       router.push("/login");
24       return;
25     }
26
27     try {
28       const user = JSON.parse(savedUser);
29
30       if (user.role !== "admin") {
31         router.push("/student-dashboard");
32         return;
33       }

```

Installation/setup instructions

Step 1: Install Required Software

Before running the Hostel and Mess Management System, the required software must be installed on the development computer. The project requires **Node.js**, **npm**, **Visual Studio Code**, and **MongoDB**.

Step 2: Open the Project

The project source code is opened in **Visual Studio Code**. The project folder contains the Next.js application files, components, pages, and other required configuration files.

Step 3: Install Dependencies

Open the terminal inside the project folder and install the required project dependencies using npm.

```
npm install
```

This installs the packages required for running the Next.js application.

Step 4: Configure the Database

The application uses **MongoDB** for storing hostel management data. The database connection details are configured through the project's environment configuration.

The database is used to store information related to students, hostels, rooms, bookings, payments, complaints, leave requests, and maintenance requests.

Step 5: Start the Development Server

After installing the dependencies and configuring the database, start the Next.js development server using:

```
npm run dev
```

The application can then be accessed through the local browser using:

```
http://localhost:3000
```

Step 6: Login to the Application

The login page provides fields for **Email** and **Password**. After entering valid credentials, the user is directed to the appropriate dashboard.

The screenshots show two different dashboards:

- **Student Dashboard** – provides bookings, payments, complaints, leave requests, maintenance, and room information.
- **Admin Dashboard** – provides management of hostels, rooms, students, bookings, payments, complaints, maintenance, leave requests, and room allocations.

Step 7: Role-Based Access

The application checks the logged-in user's role before allowing access to dashboard functions. The provided code shows that the application reads the saved user information from localStorage and checks whether the user's role is "admin". If the user is not authorized, the application redirects them to the appropriate page.

Step 8: Verify the Application

After starting the application, the major modules should be checked to ensure that they are working correctly. The student and admin dashboards, login functionality, hostel and room management, bookings, payments, complaints, leave requests, and maintenance features can be tested.

Note: Your screenshots confirm the Next.js application running at localhost:3000, the login/student/admin interfaces, and the role-checking code. They don't show the exact MongoDB setup values or the complete project folder structure, so I haven't added specific database credentials or commands that aren't visible in your material.

Geo Tag photos with guide

